

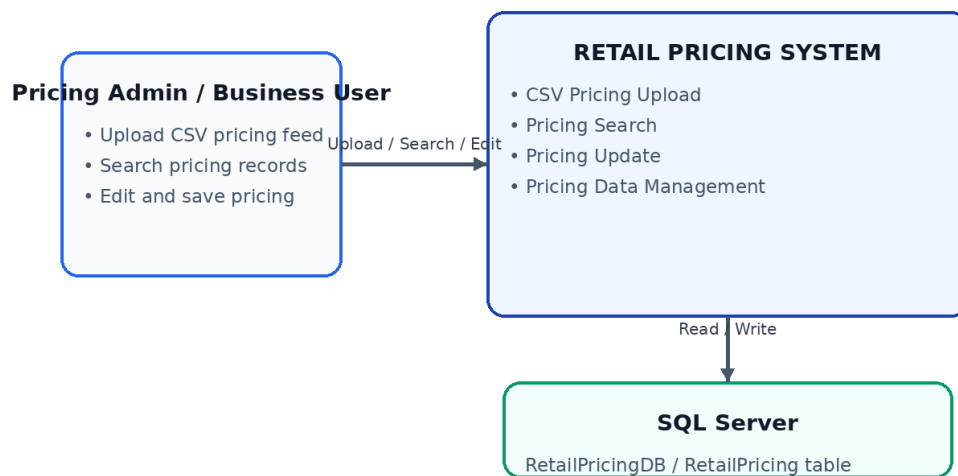
Retail Pricing Management System

Tiger Analytics Case Study — Solution Design & Implementation

Technology Stack: Angular, TypeScript, ASP.NET Core Web API (.NET 10), Entity Framework Core, CsvHelper and SQL Server.

1. Context Diagram

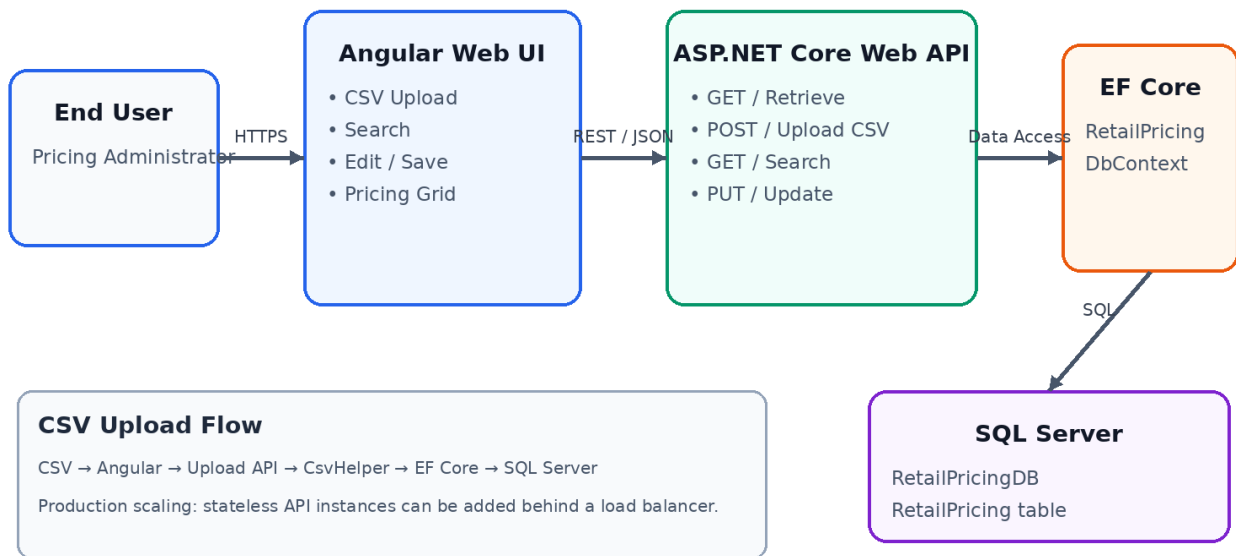
Retail Pricing — Context Diagram



The Retail Pricing System enables pricing administrators to upload pricing feeds, search pricing records and update pricing information. Pricing data is persisted in SQL Server.

2. Solution Architecture

Retail Pricing — Solution Architecture



The Angular frontend communicates with the ASP.NET Core Web API through REST endpoints. The API uses CsvHelper for CSV processing and Entity Framework Core for SQL Server persistence.

3. Design Decisions

Angular: Used for a responsive single-page application and structured component-based UI.

ASP.NET Core Web API: Provides RESTful endpoints for upload, retrieval, search and update operations.

Entity Framework Core: Provides a strongly typed and maintainable data-access layer.

SQL Server: Provides reliable relational storage and transactional consistency for pricing data.

CsvHelper: Provides reliable CSV parsing and header-to-model mapping.

REST API: Keeps frontend and backend decoupled so they can evolve independently.

Stateless API: Allows multiple API instances to be deployed for horizontal scaling.

4. Non-Functional Requirements

NFR	Requirement	Design Approach
Performance	Fast pricing searches	Efficient API queries and database indexing in production
Scalability	Support 3000 stores across countries	Stateless APIs and horizontal scaling
Availability	Keep service available	Multiple API instances and load balancing
Security	Protect pricing data	Authentication, authorization, HTTPS and secure secrets
Reliability	Avoid invalid/partial uploads	Validation and transactional persistence
Maintainability	Easy to change and extend	Separated Angular, API and data-access responsibilities
Observability	Detect failures quickly	Centralized logging, metrics and monitoring
Data Integrity	Keep pricing data consistent	Validation, database constraints and transactions

5. Assumptions

- CSV files contain Store ID, SKU, Product Name, Price and Date fields.
- Pricing ID is generated by the database.
- Authorized users can upload and modify pricing records.
- Pricing data is maintained centrally.
- Multiple countries may require currency, date-format and time-zone handling.
- Authentication and authorization will be integrated before production deployment.
- The current implementation is a functional prototype; production deployment will use scalable cloud infrastructure.
- Pricing history and audit requirements can be added if required by the business.

6. Implementation Summary

Implemented functionality includes CSV upload and persistence, pricing search, inline edit and save, responsive Angular UI, ASP.NET Core REST APIs and SQL Server persistence.

7. Source Code Repository

GitHub repository URL: **To be added after repository creation**

Recommended repository structure: frontend/ for Angular, backend/ for ASP.NET Core, and docs/ for diagrams and this case-study document.